Problem A
Secret Message Delivery
Number of test cases: 10
Time limit: 1 second

There is a mysterious spy organization that has $N = 2^k$ members. In order to prevent members' identities from being exposed, they are all represented by binary strings of k bits. For example, when $k = 2$, there are four members represented by $00, 10, 11$, and 01; when $k = 3$, there are eight members represented by $000, 100, 110, 010, 001, 101, 111$, and 011. This organization has established a protocol to send secret messages: one can send message to another if their corresponding binary strings differ in exactly one position. For $k = 3$, member 000 can send message to members 100, 010, and 001. Thus, if two spies wish to communicate, they may need the assistance of other members to serve as intermediaries and relay secret messages. Precisely speaking, if a message sending from spy x to spy y goes through t intermediaries, the message takes $t + 1$ hops. We define $d(x, y)$ to be the minimum number of message hops between spies x and y for passing one message. For example, if $x = 010$ and $y = 111$, then x can send message to 110, and then to 111. Hence, $d(x, y) = 2$ that means two hops are enough. Moreover, the *delivery range* R is defined to be $R = \max_{x,y} d(x, y)$, where x and y are two different members in the corresponding organization. For example, the delivery range of $N = 8$ is 3. Notice that each member is represented by a binary string of 3 bits in this case. If $x = 000$ sends a secret message to $y = 111$, it is not difficult to see that 3 hops (i.e. $000 \to 001 \to 011 \to 111$) are required. This idea can be applied to any x, and thus we can conclude $R = 3$. Given $N = 2^k$, your task is to write a computer program to compute the delivery range R in a spy organization with N members.

Input File Format

The first line of the input file contains an integer q, denoting the number of q test cases to follow. There are following q lines representing q test cases in which an integer N is given for each test case. Note that N is always the power of 2 and $1 \leq N \leq 2^{30}$.

Output Format

For each test case, output the delivery range R in a spy organization with N members.

Sample Input

```
2
```

```
2
2048
```

Output for the Sample Input

```
1
11
```

Problem B
Sweepers
Number of test cases: 8
Time limit: 1 second

You are given n locations along a straight road, where sensors have been installed to monitor traffic. Each location is represented as a point on the number line. To ensure the sensors are continuously monitored, k mobile units (sweepers) will be deployed.

Each sweeper moves along a fixed interval (a segment on the number line) and patrols back and forth at a constant speed of 1 unit per minute. A sensor is said to be *covered* if it is visited repeatedly by some sweeper on its patrol route.

If a sweeper patrols between two sensor locations a and b (with $a < b$), then every sensor in this interval is visited every $2 \cdot (b - a)$ minutes (moving from a to b and back). A sensor located at a single point (i.e., both endpoints are the same) is revisited every 0 minutes.

Your goal is to assign exactly k sweepers to cover all n sensors, such that:

- Each sweeper patrols a continuous group of consecutive sensor positions.
- All n sensors are covered.
- The **maximum time between two visits** to any sensor is as small as possible.

This maximum time is referred to as the **minimum possible idleness**.

- The first line contains a single integer n ($1 \leq n \leq 10^6$)—the number of sensors.
- The second line contains n space–separated integers $p_1, p_2, \ldots, p_n$ in strictly increasing order ($0 \leq p_i \leq 10^9$)–the position of the sensors.
- The third line contains a single integer k ($1 \leq k \leq n$)—the number of sweepers.

Output Format

Print one integer—the minimum possible idleness.

Examples

Input:

```
5
1 3 6 8 10
2
```

Output:

```
8
```

Input:

```
6
1 2 5 6 7 10
3
```

Output:

```
4
```

Input:

```
4
1 3 6 10
2
```

Output:

```
8
```

Problem C
Decode Messages
Number of test cases: 34
Time limit: 1 second

The NCPC code is an encryption scheme defined as follows:

1. The secret key is a randomly chosen prime number $p > 2$.
2. The plaintext (i.e., the message to be encrypted) consists of integers x satisfying $1 \leq x \leq (p-1)/2$.
3. The encryption function maps x to a ciphertext c by $c = x^2 \bmod p$.

An example of the NCPC encryption scheme is given as follows. Let $p = 7$. The set of plaintexts messages is $\{1, 2, 3\}$. Encrypting $x = 3$ gives $3^2 \bmod 7 = 9 \bmod 7 = 2$.

Given a ciphertext c, decryption in NCPC requires recovering the plaintext x such that $x^2 \bmod p = c$. This requires computing a *modular square root*. To see how this works, let us first examine the squares modulo $p = 7$:

$$\begin{array}{lll} 1^2 \bmod 7 = 1, & 2^2 \bmod 7 = 4, & 3^2 \bmod 7 = 2, \\ 4^2 \bmod 7 = 2, & 5^2 \bmod 7 = 4, & 6^2 \bmod 7 = 1. \end{array}$$

Thus the numbers $1, 2, 4$ have square roots modulo 7, while $3, 5, 6$ do not.

The numbers that have modular square roots are called *quadratic residues*, while those that do not are called *quadratic non-residues*. We write

$$a \equiv b \pmod{p}$$

to mean that a and b leave the same remainder when divided by p. For example, $3 \equiv 10 \pmod 7$, since both reduce to 3 mod 7.

Moreover, if $x^2 \equiv c \pmod p$, then $(p-x)^2 \equiv c \pmod p$ as well. For instance, with $p = 7$ and $c = 4$:

$$2^2 \bmod 7 = 4 \bmod 7$$

and

$$(7-2)^2 \bmod 7 = 25 \bmod 7 = 4 \bmod 7.$$

Hence both 2 and 5 are square roots of 4 modulo 7. They are candidates for the plaintext x. However, since the NCPC plaintext space is restricted to $1 \leq x \leq (p-1)/2 = 3$, the correct decryption of $c = 4$ is $x = 2$.

A key tool in decrypting the NCPC code is Euler's Criterion, which determines whether a ciphertext c has a modular square root. For a prime $p > 2$ and integer c with $\gcd(c, p) = 1$:

$$c^{(p-1)/2} \equiv \begin{cases} 1 \pmod p & \text{if } c \text{ is a quadratic residue,} \\ -1 \pmod p & \text{if } c \text{ is a quadratic non-residue.} \end{cases}$$

For $p = 7$, $(p-1)/2 = 3$:

$$1^3 \equiv 1 \pmod 7 \quad 2^3 \equiv 1 \pmod 7 \quad 4^3 \equiv 1 \pmod 7$$
$$3^3 \equiv 6 \pmod 7 \quad 5^3 \equiv 6 \pmod 7 \quad 6^3 \equiv 6 \pmod 7$$

Note that $6 \equiv -1 \pmod 7$. Hence $1, 2, 4$ are quadratic residues and $3, 5, 6$ are quadratic non-residues.

When c is a quadratic residue and the prime satisfies $p \equiv 3 \pmod 4$, a square root of c modulo p can be computed efficiently as

$$c^{(p+1)/4} \mod p.$$

This can be verified as follows:

$$c = c \cdot 1 = c \cdot c^{(p-1)/2} = c^{(p-1)/2+1} = c^{(p+1)/2} = \left(c^{(p+1)/4}\right)^2.$$

Since $(p+1)/4$ is an integer whenever $p \equiv 3 \pmod 4$, $c^{(p+1)/4} \mod p$ is indeed produces a valid square root of c.

Write a program that, given a secret key p and a ciphertext c, computes the corresponding plaintext x such that

$$x^2 \equiv c \pmod p, \qquad \text{with} \quad 1 \leq x \leq (p-1)/2.$$

If no such x exists, the program should output 0.

Input Format

The input data file may contain many test cases. Each test case is given in one line. The last line of the input file contains only an integer 0 to indicate the end of the input.

Each test case contains a list of integers:

1. The first integer is the secret key p, $p < 2^{32}$.
2. It is then followed by some integers c, $1 \leq c < p$. There are at most 20 c's in each test case.
3. The last integer in the list is 0, indicating the end of c's.

Output Format

To ensure that the algorithm works, your program must follow these steps:

1. Check if the key p is an odd prime (i.e., a prime number ≥ 3). If not, print "p is not a prime." and proceed to the next test case.
2. If p is an odd prime but $p+1$ is *not* divisible by 4, print "$p+1$ is not divisible by 4." and proceed to the next test case.

3. For each test case with a valid secret key p, read all ciphertexts c on that line, decode them one by one, and print the results on a single line. If a ciphertext c cannot be decoded, print "0". Adjacent output data should be separated by one space.

Sample Input and Output

Sample Input	Output for the Sample Input
7 1 2 3 4 5 6 0	1 3 0 2 0 0
13 4 12 7 0	13+1 is not divisible by 4.
11 1 2 3 4 5 6 7 8 9 10 0	1 0 5 2 4 0 0 0 3 0
24 4 7 0	24 is not a prime.
0	

Problem D
Bicycles
Number of test cases: 10
Time limit: 3 seconds

We want to ship bicycles with the leftover capacity in containers. A bicycle consists of a frame and two wheels. The weight of a frame is f, and the weight of a wheel is w. We also have n containers, with the remaining weight $c_1, \ldots, c_n$ respectively. How do we ship as many bicycles as possible in these containers? Note that a frame is in a package which must be shiped with a single container. Similarly a wheel is also in its own package and must be shiped with a single container. Finally, note that we can ship the frames and wheels anyway we want as long as they are within the leftover weight capacity of containers. That is, the frame of a bicycle does not need to be in the same container as its wheels, and one wheel of a bicycle does not need to be in the same container as the other wheel.

Input File Format

The first line has the number of test cases m. Each of the next m lines is a test case. It starts with f and w, then the number of containers n, then the leftover weights $c_1, \ldots, c_n$.

- f is an integer between 10 and 1000 inclusively.
- w is an integer between 3 and 1000 inclusively.
- n is an integer between 1 and 500 inclusively.
- c_i's are integers between 1 and 1001 inclusively.

Output Format

There are m line of outputs. Each line corresponds to the maximum number of bicycles the containers can carry.

Sample Input

```
2
3 1 4 9 2 2 2
7 1 3 7 1 1
```

Output for the Sample Input

```
3
1
```

Problem E
Counting Strings
Number of test cases: 100
Time limit: 1 second

Given an array (or set) A of N distinct elements, a *permutation array* of A is any array that is a reordering of the elements of A. If we allow each element to appear more than once, then we call it the *frequent permutation array*. In this problem, we consider strings that are related to the *frequent permutation array*.

Let S be a set of ℓ distinct symbols. For a symbol $s \in S$ and a positive integer e, we use s^e to denote a string of length e consisting of only the symbol s. We are interested in length-n strings that can be represented as $s_1^{e_1} s_2^{e_2} \cdots s_m^{e_m}$, where $s_i \in S$ satisfies $s_i \neq s_{i+1}$ for $1 \leq i < m$, and e_i's are positive integers satisfying $\sum_{i=1}^{m} e_i = n$, $1 \leq e_i \leq k+1$, $n = m+k$ and $0 \leq k < n$. For example, given $S = \{\text{a, b}\}, n = 4, k = 1$, thus $m = 3$, there are 6 possible strings:"aaba", "abba", "baab", "bbab", "abaa" and "babb".

(**Hint:** To compute the number of strings, you can first determine the possible symbols for $s_1, s_2, \cdots, s_m$ and then the possible values of e_i's.)

Given n, ℓ, k, we want to find all possible strings. Your task is to write a program to find the number of all possible strings satisfying the above requirements. Since the number can be very large, you only need to output it modulo $10^9 + 7$.

(**Hint:** To implement efficiently, you may compute and store the factorials and their multiplicative inverses modulo $10^9 + 7$ in tables.)

Input File Format

The first line in the input file has a positive integer T (≤ 100), which indicates there are T test cases. Then, for each case, there are three integers n, ℓ, and k, where $1 \leq n, \ell \leq 10^5$ and $0 \leq k < n$.

Output Format

For each test case, print the number of all possible strings in one line. Since the number can be very large, you should print the number modulo $10^9 + 7$.

Sample Input

```
3
7 2 0
5 2 1
4 2 2
```

Output for the Sample Input

```
2
8
6
```

Problem F
No Cycles
Number of test cases: 13
Time limit: 1 second

A *phylogenetic network* is a directed acyclic graph that represents evolutionary relationships among species. Peter is a biologist. He is interested in the reconstruction of phylogenetic networks. Recently, Peter encounters a problem as follows. Let $G = (V, E)$ be a directed graph in which for every pair of distinct vertices $u, v \in V$, there exists at least one edge between u and v (i.e., at least one of (u, v) and (v, u) exists). Peter wants to know whether it is possible to make G acyclic by removing a single vertex. More specifically, for a vertex $v \in V$, define $G - v$ as the graph obtained from G by removing v and all its incident edges. The problem is to find all vertices $x \in V$ such that $G - x$ contains no cycles. Please write a program to help Peter.

Technical Specification

- The number of vertices, $|V|$, is at least 3 and at most 6000.
- For every pair of distinct vertices $u, v \in V$, there exists at least one edge between u and v; and there are no self-loops, where a self-loop is an edge that connects a vertex to itself.

Input File Format

The first line contains an integer n, indicating that $V = \{1, 2, 3, \cdots, n\}$. Then, n lines follow, each of which is a zero-one string of length n. The jth character of the ith string is '1' if there is a directed edge from vertex i to vertex j, and is '0' otherwise, where $1 \leq i, j \leq n$.

Output Format

For each test case, if there is no solution, print 0; otherwise, print two integers k and $x_{\min}$, where k is the number of vertices $x \in V$ such that $G - x$ is a directed acyclic graph and $x_{\min}$ is the smallest such vertex. Print a space between k and $x_{\min}$.

Sample Input 1

```
3
011
001
000
```

Output for the Sample Input 1

```
3 1
```

Sample Input 2

```
3
011
101
110
```

Output for the Sample Input 2

```
0
```

Problem G
Sunshine Daily
Number of test cases: 7
Time limit: 2 seconds

The National Condominium Planning Company (NCPC) is planning to build a series of condominiums along a major east-west road in the central area of the city. All condos share the same footprint, with fixed width and length, but they may differ in height.

Due to urban development regulations, buildings cannot be excessively tall if doing so would result in too much shading. In particular, the Department of Urban Development requires that at least α **percent** of daylight must reach the living floors of each condo. For simplicity in computing α, assume that sunlight rays originate from a source infinitely far away. Throughout the day, the sun appears to move **at a constant rate** from the eastern horizon to the western horizon, following a semicircular arc across the sky.

Any floor that fails to receive sufficient sunlight will be designated as a parking level instead of living space. At any time point, a floor is considered **sunlit** if either its bottom east end or bottom west end receives direct sunlight. See Figure 1 for an illustration.

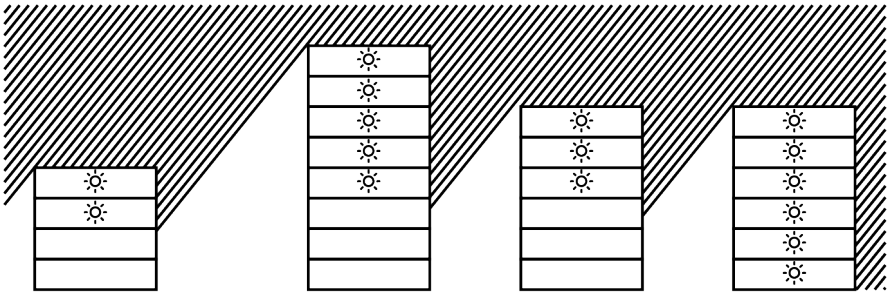


Figure 1: An illustration of several condominium buildings and the sunlight status of their floors. Roughly at 10:00AM, October 12$^{\text{th}}$, the solar altitude is about 50°. At this particular moment, the sunlit floors are marked with the sun icon. The four condo illustrated above have building locations $(x_1, x_2, x_3, x_4) = (1, 10, 17, 24)$, and their heights are $(h_1, h_2, h_3, h_4) = (4, 8, 6, 6)$.

There are n proposed building sites. Each site is described by a pair (x_i, h_i):

- x_i is the position of the building along the road (the building spans the interval $[x_i, x_i + 4]$),
- h_i is the planned height of the building in floors (each floor is 1 unit tall).

Among these n sites, m ($m \geq \max(0, n-60)$) buildings are guaranteed to be constructed. The company may freely choose whether to build on the remaining $n - m$ sites, resulting in 2^{n-m} possible building plans.

You would like to purchase an entire floor to renovate into a cozy home. Naturally, you do not want to buy a floor that could end up designated as a parking level. However, the uncertainty in the building plans makes the decision tricky.

To help clearing up your mind, a parameter K is defined. A floor in building i is considered **legit** to acquire if there are at least K different building plans such that:

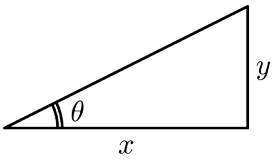
1. building i is constructed in the plan, and
2. the floor is sunlit for $\geq \alpha\%$ of the daylight time in *all* those K plans.

Your task is to determine, for each building, **how many of its floors are legit** to acquire.

Input File Format

The input contains multiple test cases. Each test case consists of 3 lines:

- **First line**: four integers n ($1 \leq n \leq 500$), m ($\max(0, n-60) \leq m \leq n$), K ($1 \leq K \leq 2^{n-m}$), and α ($1 \leq \alpha \leq 99$). For resolving technical issues and confusions, the input further gurantees that $\tan(\alpha\pi/100)$ is **an irrational value**. That is, $\alpha \notin \{25, 50, 75\}$. Here is a simple picture depicting the value to the trigonometric function $\tan\theta$ given an angle θ in radians:



$$\tan\theta = \frac{y}{x}$$

- **Second line**: $2n$ integers in the order $x_1, h_1, x_2, h_2, \ldots, x_n, h_n$. For all i, $1 \leq x_i, h_i \leq 10^9$. For each i, $2 \leq i \leq n$, it is guaranteed that $x_i \geq x_{i-1} + 5$.
- **Third line**: m integers $t_1, t_2, \ldots, t_m$ ($1 \leq t_1 < t_2 < \ldots < t_m \leq n$), denoting the indices of buildings that are guaranteed to be built.

The input terminates with a line containing a single integer 0.

Output Format

For each test case, output n integers $s_1, s_2, \ldots, s_n$ in a single line, where s_i denotes the number of floors in building i that are **legit** to acquire. The values must be separated by a single whitespace.

Sample Input

```
4 4 1 99
1 4 10 8 17 6 24 6
1 2 3 4
4 4 1 70
1 4 10 8 17 6 24 6
1 2 3 4
4 2 2 49
1 4 10 8 17 6 24 6
1 3
4 0 7 49
1 4 10 8 17 6 24 6

0
```

Output for the Sample Input

```
0 2 0 0
2 4 0 4
4 7 6 6
4 7 2 6
```

Problem H
Cracking
Number of test cases: 12
Time limit: 1 second

Two countries, NCPC and CPCN, are at war. CPCN is using a highly unique method to encode its messages and obfuscate their content, while NCPC is trying to decipher CPCN's encoding. You are the Director of NCPC's Information Security Department, and you've learned that CPCN employs the following methods for encryption. Let x be the original message, consisting of English letters (lowercase or uppercase), and x is always nonempty.

1. Concatenate twice of x to form a string s_1 (i.e. $s_1 = xx$).
2. Split s_1 into two substrings y and z (either may be empty) in some unknown way, such that $s_1 = yz$.
3. Generate an arbitrary string w (which may be empty), and form $s_2 = ywz$.
4. Output s_2 as the cipher text.

For example, set x as `attackAtDawn`. Then, you get s_1 as `attackAtDawnattackAtDawn`. Suppose we split s_1 into `attackAt` as y and `DawnattackAtDawn` as z, and set w as `moonNotAt`. Finally, the cipher text s_2 would be `attackAtmoonNotAtDawnattackAtDawn`.

Write an efficient program to recover all possible values of x from the given input s_2. Since multiple valid values of x may exist, output the total number of distinct solutions of x as well as the length of the longest valid x in the same line, with a space as the delimiter.

Input File Format

The first line gives you a positive integer t that specifies the number of test cases. In the following t lines, each line has a string served as one test case that you need to crack. The total length of all strings in the input file is less than 2×10^6.

Output Format

The output of each test case occupies exactly one line, where the first number is the total number of distinct solutions and the second number is the length of the longest solution.

Sample Input

```
4
abxab
aaBaaBaaB
aaabbaaabbbaabbb
ncpcistoCPNCisfulloflaughncpcistough
```

Output for the Sample Input

```
1 2
2 3
4 5
1 11
```

Remarks. We use $[i, j]$ to denote a substring starting from index i to index j. In the first test case, x is `ab` and w is $[2, 2]$. In the second test case, x could be `a` (with $w = [2, 8]$) or `aaB` (with $w = [3, 5]$). In the third test case, x could be `a` (with $w = [2, 15]$), `b` (with $w = [0, 13]$), `aaabb` (with $w = [8, 13]$), or `aabbb` (with $w = [2, 7]$). In the forth test case, x is `ncpcistough` with $w = [8, 21]$.

Problem I
A Biologist's Task
Number of test cases: 30
Time limit: 1 second

Alice is a biologist who is working on the evolutionary history of a given set V of species. She has the evolutionary distances between some pairs of species, represented by an edge-weighted graph $G = (V, E)$ with edge-weight function w, where $w(u, v)$ for any $(u, v) \in E$ stands for the distance between the two species u and v.

The evolutionary history is visualized by an evolution tree. An evolution tree is described by a rooted tree $T = (U, E_T)$ and an edge-weight function d for E_T such that the following conditions are satisfied.

1. The set of leaf nodes in T is exactly the set of species in V, i.e., for any $v \in U$, $\deg_T(v) = 1$ if and only if $v \in V$, where $\deg_T$ is the degree function for T.
2. $\deg_T(r) \geq 2$ for the root node r of T, and $\deg_T(v) \geq 3$ for any non-root internal node $v \in U$.
3. The distances between the root node r and any leaf node are the same, i.e.,

   $$d_T(r, u) \;=\; d_T(r, v)$$

   for any two leaf nodes $u, v \in V$, where d_T denotes the total weight of the path between the given nodes in T.

Alice is interested in finding an evolution tree that best describes the evolution distances she has for V. Precisely speaking, if T is an evolution tree with edge-weight function d, then the quality of the tree, i.e., how well it fits the given evolutionary distances, is measured by the maximum pairwise difference, i.e.,

$$\Delta(T, d) \;:=\; \max_{(u,v)\in E} \left| w(u, v) - d_T(u, v) \right| .$$

Given the evolutionary distances Alice has for V, please help her fulfill her task and find an evolution tree (T, d) such that $\Delta(T, d)$ is minimized.

Input File Format

The first line of the input contains two integers n and m, where n is the number of species in V and m is the number of edges in E. Then there are m lines, where the i-th line contains three integers u_i, v_i, and w_i indicating that (u_i, v_i) is an edge in E with a weight of w_i. You may assume that

- $2 \leq n \leq 10^3$ and $1 \leq m \leq 10^3$.

- The species are indexed between 1 and n.
- $1 \leq u_i, v_i \leq n$ and $u_i \neq v_i$ for all $1 \leq i \leq m$.
- $1 \leq w_i \leq 10^9$ for all $1 \leq i \leq m$.
- $(u_i, v_i) \neq (u_j, v_j)$ for all $1 \leq i, j \leq m$ with $i \neq j$.

Output Format

Output an evolution tree T with an edge-weight function d such that $\Delta(T, d)$ is minimized. In the first line print two integers k and r, the total number of nodes in the tree T and the index of the root node. In each of the next $k-1$ lines, print two integers x_i, y_i, and one non-negative floating point number d_i, indicating that there is an edge between x_i and y_i with weight d_i in the tree.

Note that, your output must follow the following guidelines.

- Use the same indexes for the leaf nodes as used in Alice's description.
- Label all nodes with indexes between 1 and k.

As there may be a multiple number of evolution trees that best describes the evolutionary distances of the species, Alice is happy with any one of them. This says, if there are multiple answers, you may print any one of them. Furthermore, we accept solutions that have an absolute error at most 10^{-6}.

Sample Input 1

```
3 3
1 2 10
2 3 18
1 3 18
```

Sample Output 1

```
5 5
1 4 5
2 4 5
4 5 4
3 5 9
```

Sample Input 2

```
3 3
1 2 2
1 3 4
2 3 6
```

Sample Output 2

```
5 5
1 4 0.5
2 4 0.5
4 5 2
3 5 2.5
```

Problem J
Twin Shields on Planet X-1012
Number of test cases: 3
Time limit: 6 seconds

During the colonization of Planet X-1012, humanity has discovered multiple energy mining sites. Each site requires a circular protection zone to ensure operational safety and shield it from unstable energy emissions. To conserve rare construction materials, the engineering division will deploy at most two circular protective shields, each of adjustable size and position, to cover all mining zones. The objective is to minimize the size of the larger of these two shields while ensuring every mining zone is fully covered by at least one shield. Note that the protection zones of different mining sites may overlap with each other.

You are given N mining zones. Each zone is represented by a circle defined by its center coordinates and radius. Your task is to determine the minimum possible value of the larger radius between the two protective shields needed to completely cover all zones, such that each given circle lies entirely inside at least one shield.

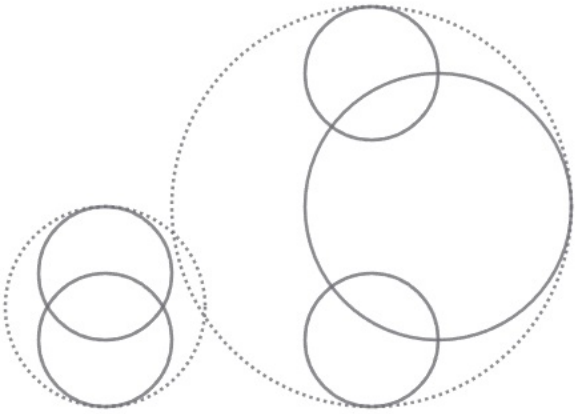


Figure 1

For example in Figure 1, there are five zones described by (x_i, y_i, r_i) for the i^{th} zone where x_i, y_i are its center coordinates and r_i is its radius. They are $(0, 0, 1)$, $(0, 1, 1)$, $(4, 0, 1)$, $(4, 4, 1)$, and $(5, 2, 2)$. Two protective shields could be $(0, 0.5, 1.5)$ and $(4, 2, 3)$ where $(0, 0.5, 1.5)$ covers the first two zones, and $(4, 2, 3)$ covers others.

Input File Format

The input contains N lines ($1 \leq N \leq 200$). Each line describes one zone (the i^{th} zone) and contains three integers x_i y_i r_i, separated by a single space. $0 \leq x_i, y_i \leq 10^4$ are the coordinates of the zone center. $0 < r_i \leq 10^4$ is the required protection radius.

Output Format

Print a single line containing the minimum possible radius of the larger shield. Please round the answer to four decimal places.

Sample Input

```
0 0 1
0 1 1
4 0 1
4 4 1
5 2 2
```

Output for the Sample Input

```
3.0000
```

Problem K
Light Sticks
Number of test cases: 15
Time limit: 2 seconds

Alice is decorating a concert venue using LED light sticks. She has placed exactly m light sticks, and there are n available **attachment points** in total. Each light stick is attached to **two distinct attachment points** chosen from this pool of n points. For every pair of attachment points, there is **at most one** light stick connecting them. Each light stick can be set independently to one of k colors, numbered $1, 2, \ldots, k$, or turned off.

After completing the placement of the light sticks, Alice realizes that she was not supposed to arrange them arbitrarily. Instead, she has been asked to display k given **patterns**, each specified as a tree. Specifically, for each $i \in \{1, 2, \ldots, k\}$, a tree $T_i = (V_i, E_i)$ is provided. Alice must assign colors to the light sticks so that, for each i, the set of light sticks set to color i induces a graph that is **isomorphic** to T_i.

Formally, let U_i be the set of attachment points connected by light sticks assigned color i. We require that there exists a bijection $f : V_i \to U_i$ such that for every pair $x, y \in V_i$,

$$\{x, y\} \in E_i \quad \text{if and only if} \quad \{f(x), f(y)\} \text{ is connected by a light stick of color } i.$$

Your task is to write an efficient program to determine whether Alice can assign colors to the light sticks so that all k patterns are realized as described.

Input File Format

Each test case consists of the following:

- A line containing three integers n, m, and k: the number of attachment points, light sticks, and patterns, respectively.

$$4 \le n \le 500, \quad \min\{2(n^{1.5}), n(n-1)/2\} \le m \le n(n-1)/2, \quad 1 \le k \le n.$$

- The next m lines each describe a light stick with two integers u and v ($1 \le u \ne v \le n$), indicating that a light stick is attached between attachment points u and v. No unordered pair $\{u, v\}$ appears more than once.
- Then, for each $i = 1$ to k, the description of tree $T_i = (V_i, E_i)$ is given in the following format:
  - A line with a single integer $t_i = |V_i|$ where $2 \le t_i \le \sqrt{n}$ denotes the number of nodes in pattern i (i.e., the number of nodes in tree T_i). It is guaranteed that T_i is a tree (i.e., has exactly $t_i - 1$ edges).
  - Then $t_i - 1$ lines follow, each containing two integers a and b ($1 \le a \ne b \le t_i$), denoting an undirected edge between nodes a and b in tree T_i.

Output Format

If a valid coloring exists, output two lines for each $i \in \{1, 2, \ldots, k\}$ describing the tree T_i. The first line contains a single integer t_i, the number of nodes in T_i. The second line contains exactly t_i integers $a_1, \ldots, a_{t_i}$, where for each $j \in \{1, 2, \ldots, t_i\}$ the j-th node of T_i is mapped to the a_j-th attachment point. If there are multiple solutions, output any of them.

Otherwise, if no coloring satisfies the requirements for all k trees, output a single line containing `impossible`.

Sample Input

```
4 6 2
1 2
1 3
1 4
2 3
2 4
3 4
2
1 2
2
1 2
```

Output for the Sample Input

```
2
4 1
2
4 2
```

Problem L
Viral Marketing
Number of test cases: 9
Time limit: 2 seconds

There are n people, denoted 0, 1, ..., $n-1$. For any distinct people $i, j \in \{0, 1, \ldots, n-1\}$, i is a friend of j if and only if j is a friend of i.

For all $i \in \{0, 1, \ldots, n-1\}$, i is not a friend of i. Any two distinct people are connected by a chain of friendships. Bob wants to promote a product to all people. In round zero, he gives free products to a set $S \subseteq \{0, 1, \ldots, n-1\}$ of people. Since then, everyone in S has the product. In each subsequent round, a person not yet having the product will buy the product if more than half of his/her friends already have the product in the previous round. Once someone has the product, (s)he will have it forever.

Call S a *dynamic monopoly* if all people have the product eventually. Furthermore, call S a *static monopoly* if all people have the product in round one (which is the round after round zero). Due to limited budget, Bob needs that $|S| \leq (n+1)/2$. Please help him find a static monopoly if everyone has exactly three friends and a dynamic monopoly otherwise.

For example, assume that $n = 8$, 0 is a friend of 6, 0 is a friend of 3, 0 is a friend of 4, 1 is a friend of 6, 1 is a friend of 3, 1 is a friend of 2, 2 is a friend of 4, 2 is a friend of 7, 3 is a friend of 5, 4 is a friend of 5, 5 is a friend of 7 and 6 is a friend of 7. Then $S = \{2, 3, 5, 6\}$ is a static monopoly of size at most $(n+1)/2$, as verified below:

- $4 = |S| \leq (n+1)/2 = 4.5$.
- In round zero, 2, 3, 5 and 6 receive the product.
- In round one, 0, 1, 4 and 7 buy the product.

For another example, assume that $n = 6$, 0 is a friend of 4, 1 is a friend of 4, 0 is a friend of 1, 0 is a friend of 5, 1 is a friend of 2, 2 is a friend of 3, 2 is a friend of 5 and 3 is a friend of 4. Then $S = \{0, 2\}$ is a dynamic monopoly of size at most $(n+1)/2$, as verified below:

- $2 = |S| \leq (n+1)/2 = 3.5$.
- In round zero, 0 and 2 receive the product.
- In round one, 1 and 5 buy the product.
- In round two, 4 buys the product.
- In round three, 3 buys the product.

Input File Format

The first line is the number of test cases. Each test case is given as follows. The first two lines are n and the number of friendships. The remaining lines specify the friendships. In particular, if i is a friend of j, then there is exactly one line consisting of i and j (or j and i), where $i, j \in \{0, 1, \ldots, n-1\}$. The number of friendships is at most 1199970, and $n \leq 999999$.

Output Format

For each test case, find a static (dynamic) monopoly S of size at most $(n+1)/2$ if everyone has exactly three friends (someone's number of friends differs from three, respectively). Print the elements of S in ascending order followed by -1. If there are two or more correct solutions, please just output one of them.

Sample Input

```
2
8
12
0 6
0 3
0 4
6 1
1 3
1 2
2 4
2 7
3 5
4 5
5 7
6 7
6
8
0 1
0 4
0 5
1 2
1 4
2 5
2 3
3 4
```

Output for the Sample Input

```
2 3 5 6 -1
0 2 -1
```